

TECHNICAL SUMMARY SHEET

The free C-Program API Library is easy to use and fail-safe for applications to generate log messages.

This C-Program Library is free to use under the MIT License agreement. It provides a simple printf() style API function interface to generate application log messages:

<code>LoggerAppsMsg(DevProd, LogicalName, SeverityLevel, "Format string specification, string specifier value %s", "string value");</code>
--

Technical Advantage:

The primary benefit using this API library over application loggers with similar capabilities, is that it does extensive runtime validation to prevent the application from crashing due to common mis-use of printf() style issues (e.g., passing %s a NULL value, invalid NULL terminated string value). If a runtime validation fails the API function generates an error code and simply returns, allowing the application to continue normal processing.

API Usage:

1. DevProd controls generating development and production log messages.
2. LogicalName indicates file name, function name or common name across multiple files.
3. SeverityLevel indicates different levels of severity importance.
4. The remainder of the arguments are similar to printf() style specification.
5. API Arguments Validation:
 - a. Use only LogicalName defined for use by the API.
 - b. Use only SeverityLevel defined for use by the API.
 - c. Decode printf() format string specification for string specifiers and match to corresponding arguments.
6. API Bound Argument Validation:
 - a. Validate length of format string specification to prevent use of unbounded strings.
 - b. Validate length of string argument to prevent the use of improper NULL terminated strings.
 - c. Validate specific ASCII characters to avoid use of unwanted characters in log message.
 - d. Validate against an acceptable set of format string specifiers to prevent unsafe and malicious use of attacker string specifiers.
 - e. Validate LogicalName and SeverityName for allowed valid use.

Explanation on how the Bound works is provided in Wandl-SSAL-Bound-Capabilities.pdf and detailed examples on how the Bound Argument Validation works are provided in the Wandl-SSAL-examples.c program.

7. Other Preventions:
 - a. Invalid NULL, 0, -1 pointer for %s format string specifier.
 - b. Untrusted or malicious format string specifiers.
 - c. Ill-formed or non-standard format specifiers.
8. Robust and Fail-Safe API:
 - a. Application does not need to check API return status.
 - b. Most fail-safe occurrence will not impact application execution flow.
9. Standard log file name includes: [date.timestamp.sequence-number]
10. Standard output log file content format:

<code>02/09/2026 12:01:55 [3650:sample_program] [DATABASE] [INFO] Message that generates 1 string and 2 integer arguments [String Argument], [98] and [99]</code>

where:

"02/09/2026 12:01:55" date and timestamp when the log message was generated.

"[3650:sample_program]" application process id and name.

"[DATABASE]" logical message group for related application log messages.

" [INFO]" severity for log message.

The remainder of the output is derived from the C-Programming Language printf() format string specification and corresponding arguments.

Contact: bwoodley@istech.com

Download: <https://github.com/wandi-ssal/secure-application-logger>